Poool Game

Concurrent and Distributed Programming
a.y. 2025–2026

Buda Marco Merighi Daniele Turchi Jacopo

June 25, 2026

1 Problem Analysis

This section provides an analysis of the `Poool` game from a concurrent programming perspective. The main computational challenge resides in the simulation loop, which must continuously handle physics updates, boundary collisions, and elastic collisions among thousands of small balls.

The primary bottlenecks identified are:

- **Linear State Updates:** Modifying positions and applying friction coefficients across large arrays of entities.
- $O(n^2)$ **Collision Resolution:** The nested loop required to evaluate and resolve simultaneous impacts between every pair of balls represents the highest density of CPU cycles.

2 System Architecture and Architectural Design

This section describes the high level structure of our proposed solution, focusing on the elements of concurrent programming, including the active and passive components at play and their interactions. Visual formalisms, specifically Petri nets, are also used to demonstrate the overall behavior and to allow for basic model checking.

2.1 Preliminary Optimizations

Before considering a multi-threaded approach, it is possible to improve the collision resolution algorithm significantly. The proposed change will boost the overall performance and, most importantly, will allow for better parallelization efficiency. As this optimization is also applicable to the sequential version, the performance evaluation will take the improved algorithm as the starting point.

Instead of checking for collisions between every pair of balls, the board is divided into a grid of cells such that collisions may only happen within neighboring cells. Although this

method introduces an additional phase for the population of the grid at every simulation step, it is linear in the number of balls and drastically reduces the multiplicative constant for the remaining quadratic part.

To be precise, for each cell, collisions are resolved for balls within the cell itself, then against the *forward-neighboring* cells (i.e. the right, down-left, down, and down-right neighbors). In this way, collisions are never resolved twice.

2.2 Parallelization Strategy

Having identified that the physics simulation represents the most taxing part of the program's computation, it will be the focus of our parallelization efforts. Since it is a CPU-bound problem, the goal will be to distribute the work evenly across all logical processors.

Our solution involves a number of worker threads, handled manually in the Thread-Oriented version, and encapsulated by a `FixedThreadPool` executor in the Task-Oriented version. We opted for long-living worker threads, implementing Agenda parallelism, thus removing any thread creation overhead.

Every simulation step consists of three phases: updating ball positions, detecting balls inside holes, and resolving collisions. The first two phases are linear in the number of balls, making it is easy to distribute the work evenly, while the collision phase requires a more careful approach.

We decided to view the problem in terms of grid rows. In the Thread-Oriented version, we distribute the rows to the different worker threads by implementing round-robin scheduling. In the Task-Oriented version, each row constitutes a task which is submitted to the executor. For a sufficiently fine grid, this makes for a quite even partition.

In both cases, we handle the rows in two successive subphases, according to their parity. Combined with the forward-neighbor strategy, this ensures that rows can be processed concurrently without ever accessing any shared memory.

2.3 Active Components

The concurrent program's execution involves instances of five active classes, each encapsulating a designated control flow.

- `KeyboardController`, responsible for the execution of issued keyboard commands, keeping the Graphical User Interface responsive.
- `BotUpdater`, periodically kicking the bot player's ball.
- `SimulationCoordinator`, distributing the computation for the physics simulation.
- `SimulationWorker`, which performs an assigned portion of the simulation workload. In the Task-Oriented version it is replaced simply by `java.lang.Thread` and managed by the Java Executor framework. In both cases, multiple instances are expected to be active during the game's runtime.

- `java.awt.EventDispatchThread`, provided by Java Swing, which handles graphical events.

2.4 Monitors

All active components are implemented to be pure, meaning that interactions between them happen through synchronized passive components. Our solution makes use of five such classes.

- `BoundedBuffer`, used as the keyboard command buffer.
- `Latch`, handling synchronization between the simulation coordinator and workers in the Thread-Oriented version.
- `SynchCell`, used to assign work to individual workers in the Thread-Oriented version.
- `AtomicReference`, used to manage the concurrent access to the game's mutable state (e.g. the player scores) and the kick velocity of player balls.
- `AtomicList`, used to manage the concurrent access to the mutable collection of balls within the game.

These monitor classes are implemented following the standard semantics, except for `SynchCell`, which features an additional `end()` method, used to signal that no more values will be put into the cell. It wakes up all threads that happen to be blocked on the `get()` method, causing an empty `Optional` to be returned.

2.5 Behavior Overview

We use Petri nets to show a simplified view of the program's information flow. At this level of abstraction the events are atomic, due to all monitor procedures being marked as `synchronized`. This also means that the mutual exclusion to these objects doesn't need to be depicted explicitly, focusing instead on the more high level constructs.

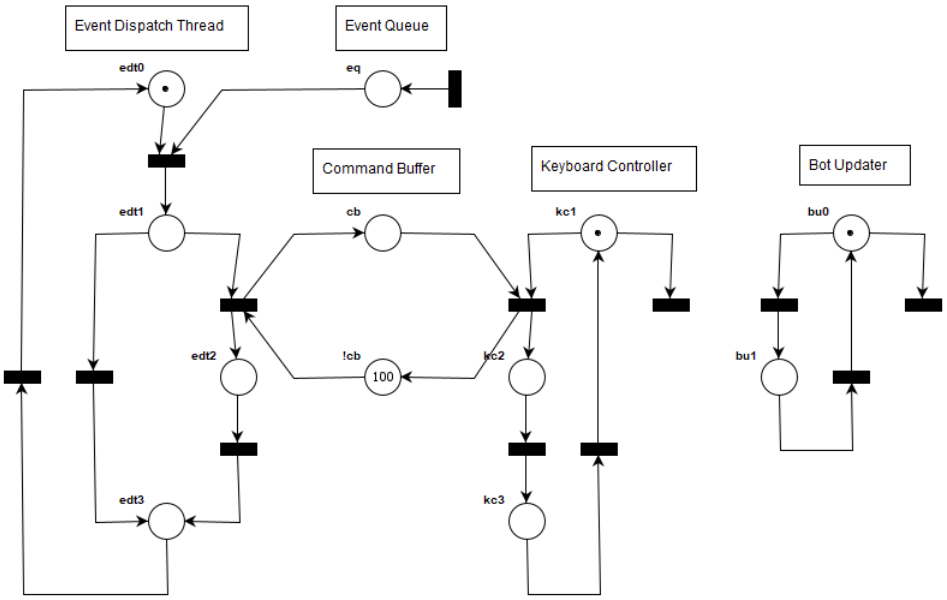

Figure 1: Petri net for the interactions between the Event Dispatch Thread, Keyboard Controller, and Bot Updater active instances.

Figure 1 shows an approximation of the control flow within Java Swing's Event Dispatcher Thread, which repeatedly gets events from the Event Queue and executes them. In the case of a keyboard event, the registered callback issues a keyboard command to the command buffer.

The Keyboard Controller follows a similar logic, getting commands from the buffer and executing them within the game's context, until the game ends.

The Bot Updater is autonomous, consisting only of a periodic interaction with the bot player's ball, likewise stopping once the game ends.

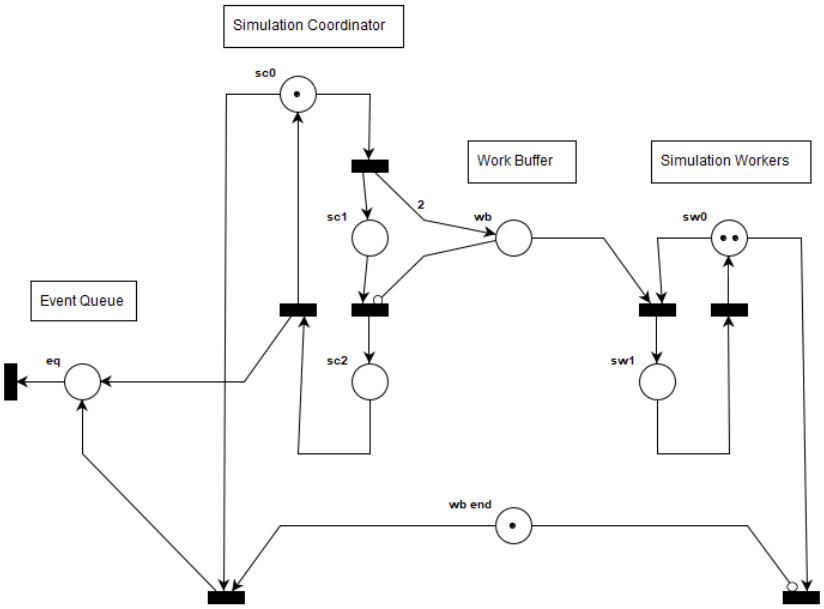

Figure 2: Petri net for the interactions between the Simulation Coordinator and Workers active instances.

Figure 2 shows how the physics simulation is parallelized, using two designated workers in this example.

At every iteration, the coordinator divides the workload and assigns it to the work buffer. In the Thread-Oriented version, this buffer is implemented as a list of synchronized cells, so that the workers are not competing for a single object, alongside a latch which is used to detect when all threads have completed their assigned work. In the Task-Oriented version, the same is achieved by making use of the `executorService.invokeAll()` method.

The workers simply wait for work on their respective cell, calling `latch.coundDown()` upon completion.

At every iteration, the coordinator may determine that the game is over. In the Thread-Oriented version, this is done by calling the `synchCell.end()` method for all workers, so that they are allowed to terminate. In the Task-Oriented version, the coordinator simply calls `executorService.shutdown()`.

As is shown, each simulation step, as well as the game's ending, generates an event to be processed by the Graphical User Interface. The event is added to the Event Queue by means of the `SwingUtilities.invokeLater()` method.

2.6 Transition Analysis

The reported Petri nets were analyzed using the Platform Independent Petri net Editor (PIPE). The reachability graph is shown to contain a single marking with no active

transition, which happens to be the state in which all threads have terminated. Because of this, we can assert that the proposed solution is correct and free of deadlocks.

3 Performance Evaluation

This section evaluates the empirical performance gains of the concurrent implementations compared to the baseline sequential approach under heavy workloads.

3.1 Methodology

Testing was conducted utilising the `MassiveBoardConf` configuration (4500 entities) to generate a compute-intensive workload. To ensure measurement accuracy, the physics and collision resolution logic `updateState()` was isolated. The GUI was disabled to eliminate the interference of the sequential Swing event dispatch thread.

Execution times were recorded using `System.nanoTime()`. Each configuration executed a 10000-update warm-up phase to allow the JVM Just-In-Time (JIT) compiler and Garbage Collector to stabilise, mitigating dynamic compilation overhead as described in "Performance of Concurrent Programs" slides. The following 20000 updates were measured, and the average median execution time was calculated across 5 independent runs per configuration.

The baseline sequential execution time (T_{seq}) averaged 3.637ms per frame for the task version and 3.727ms for the thread one. The host system for the benchmark featured 12 available logical processors (6 physical execution units with Simultaneous Multithreading) $N_{cpu} = 12$.

3.2 Concurrent vs. Sequential Advantage

To evaluate how much the concurrent version improves upon the sequential one, Speedup (S) was calculated using the formula $S = T_{seq}/T_N$, where T_N is the execution time with N workers and T_{seq} is the time with 1 worker.

Workers	Task Time (ms)	Thread Time (ms)	Task Speedup	Thread Speedup
1	3.727	3.637	1.00x	1.00x
2	2.093	2.302	1.78x	1.58x
4	1.443	1.455	2.58x	2.50x
6	1.158	1.113	3.22x	3.27x
8	**1.039**	1.006	**3.58x**	3.61x
10	1.056	**0.985**	3.53x	**3.69x**
11	1.062	1.008	3.51x	3.61x
12	1.064	1.026	3.50x	3.55x
13	1.062	1.062	3.51x	3.42x

Table 1: Average execution time and Speedup for Task-Oriented and Thread-Oriented architectures.

The empirical data demonstrate a definitive advantage of concurrent execution. The Task-Oriented implementation (utilising the Executor framework) achieves a peak speedup of 3.58x at 8 workers, reducing the frame computation time from 3.727ms to 1.039ms. The Thread-Oriented architecture yields a similar peak speedup of 3.69x (0.985ms). This confirms that distributing the physics workload across multiple active components substantially accelerates execution compared to the sequential variant.

3.3 Core Exploitation and Scalability

To assess how effectively the program exploits available cores, Efficiency (E) was calculated using the formula $E = S/N$.

Workers	Task Efficiency	Thread Efficiency
1	1.00	1.00
2	0.89	0.79
4	0.65	0.63
6	0.54	0.54
8	0.45	0.45
10	0.35	0.37
11	0.32	0.33
12	0.29	0.30
13	0.27	0.26

Table 2: Computational Efficiency scaling per worker count.

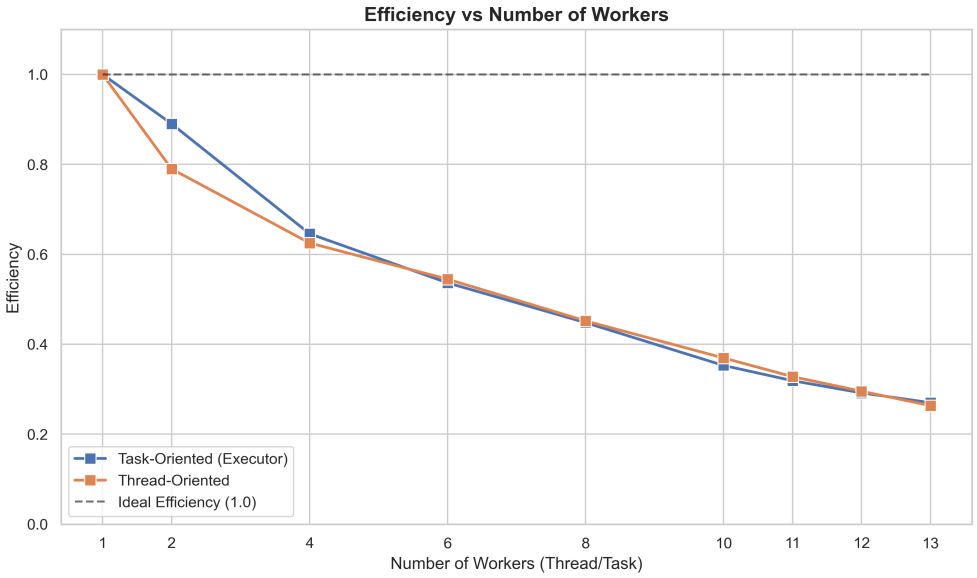

Figure 3: Efficiency decay.

The application exhibits good efficiency ($> 63\%$) when scaling from 1 to 4 workers. The computation-heavy nature of the collision detection algorithm allows the processors to be exploited directly with minimal synchronisation penalties.

However, efficiency decays as the worker count increases. At 8 workers, efficiency drops to 45%, and at the N_{cpu} limit of 12 workers, it falls to 30% (Thread-Oriented). This sub-linear scaling is caused by multithreading costs, specifically context switching and lock contention over the shared structures tracking the game state.

Overall, the Task-Oriented framework proved slightly more effectiveness at exploiting cores than the Thread-Oriented approach, maintaining lower structural overhead per unit of work distributed. At least until 8 workers, when the access of the bag of task from multiple threads might have issued the performance of that approach.

3.4 Theoretical Limits and Amdahl's Law Analysis

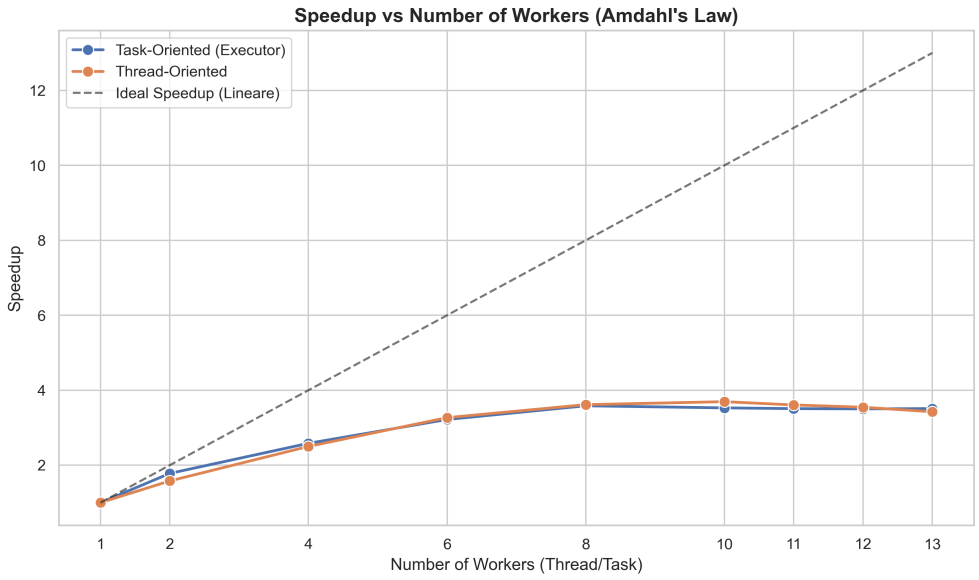

Figure 4: Speedup scaling compared to the ideal linear trajectory.

To mathematically contextualise the deviation from the ideal linear speedup shown in Figure 4, Amdahl's Law is applied. This principle states that the maximum achievable speedup is strictly bounded by the fraction of the algorithm that can be made parallel (P)

Using the peak empirical speedup ($S = 3.69$) achieved with $N = 10$ workers in the Thread-Oriented architecture, the exact sequential and parallelizable (P) proportions of the application can be derived:

$$S = \frac{1}{1 - P + \frac{P}{N}} \implies 3.69 = \frac{1}{1 - P + \frac{P}{10}}$$

Solving for $P = \frac{2.69 \times 10}{3.69 \times 9}$, it is determined that the highly parallelizable portion of the program is $P \approx 0.81$ (81%).

- **The Parallelizable** 81%**:** This vast majority of execution time is successfully distributed across the worker pool. It comprises evaluating elastic collisions, updating entity positions and resolving boundary bounces.
- **The Sequential** 19% **(The Bottleneck):** This unscalable fraction prevents the application from reaching ideal efficiency. Code analysis of the `Simulation Coordinator` isolates this bottleneck to:
  - **Barrier Synchronization Overhead:** The coordinating thread must block and wait for all workers to complete their discrete tasks via `invokeAll(work)` or `Latch.await()` before proceeding to the next frame computation step.

- **Human and Bot kicks:** The action of kicking the ball are done sequentially in the main thread (after beeing putted in queue).
- **Centralized State Evaluation:** Operations such as checking `isGameOver()` and verifying if any small balls remain are inherently sequential and execute solely on the primary thread, even if some worker would be ready for more work.
- **Grid creation:** This represents the main sequential cost. The grid needs to be cleared and then populated.

This 19% structural bottleneck accurately explains why the speedup does not grow proportionally as the number of cores increase, visually validating the theoretical limits predicted by Amdahl's Law.

4 Program Verification and Model Checking

This section reports the formal verification of the simulation logic with Java PathFinder (JPF), an explicit-state model checker that explores all the thread interleavings of a Java program. The goal was to confirm the absence of data races and deadlocks in both concurrent architectures.

4.1 Methodology and Verification Target

JPF supports only Java 11 and does not scale to arbitrary program sizes, because its state space explodes with the number of threads and synchronisation events. Direct verification of the production code was not possible. A minimal variant `pooolThreadOrientedJpf` was therefore built, faithful to `pooolThreadOriented` except where strictly required.

The view, the keyboard controller, the board observers, and the `BotUpdater` were removed to keep the state space tractable for JPF. `BotUpdater` also relied on `java.util.Random`, which JPF treats as an unbounded non-deterministic choice generator. `Record` declarations and `switch` expressions were rewritten in pre-Java 14 form. The lambdas inside `distributeWork` were replaced with anonymous inner classes, because JPF historically does not handle `invokedynamic` reliably. A bounded counter `maxTicks` was added to the coordinator to guarantee termination, as JPF requires the program to halt before it can certify any property. A constant tick interval replaced `System.currentTimeMillis()`, which under JPF does not advance.

The collision phase now uses a spatial grid. Its production size (150 cells) made even the two-ball base run run out of memory, so for JPF the cell size was enlarged with a single constant (`maxRadius` from 0.1 to 0.5), reducing the grid to 6 cells. This loses no collisions, because the balls are far smaller than a cell, and still keeps both workers busy in the collision phase.

The base configuration used `MinimalBoardConf`, two workers, and two ticks. The listener `PreciseRaceDetector` was active in addition to the default `NotDeadlockedProperty`.

4.2 Race Condition on Ball Position

The first violation raised by JPF was a data race on `Ball.pos`. The opening lines of `Ball.resolveCollision` accessed `a.pos` and `b.pos` as direct fields, while the writes always go through the `synchronized` accessors of `Ball`. Under the Java Memory Model this counts as a race, and `PreciseRaceDetector` flagged it. The fix was to read `pos` and `radius` through the existing getters at the entry of the method, so that every access happens under the same monitor as the writes. The same change was backported to the production `pooolThreadOriented` and `pooolTaskOriented`.

4.3 Shutdown Deadlock and Channel Closure

After the race fix, JPF detected a deadlock: worker threads blocked in `SynchCell.get()` with the coordinator already `TERMINATED`. The cause is a race between the worker's check of `gameState.isGameOver()` at the top of its loop and the next call to `SynchCell.get()`. When a ball falls into a hole, the worker processing it sets `gameOver = true`. The coordinator then exits its main loop and stops enqueueing work. Any other worker that had passed the `isGameOver()` check just before now blocks in `wait()`, because the only producer for that queue is gone. The defect is structural: coordinator and worker consult the same flag to decide termination, without coordination.

The fix replaced the shared-flag shutdown with a channel-closure pattern. `SynchCell` gained a `synchronized end()` method, and `get()` now returns `Optional<T>`, yielding `Optional.empty()` once closed. The worker loop became a plain consumer that exits on the empty value and no longer reads `gameState`. The coordinator calls `end()` on every synchCell as its last action, removing the race window by construction. The same change was backported to the production `pooolThreadOriented`.

4.4 Scenario-based Verification

The `MinimalBoardConf` configuration exercises only one termination branch of the coordinator, namely loop exit via `maxTicks`. To cover the remaining branches and other concurrency paths, six dedicated configurations were authored under `scenarios/`, each isolating a distinct case:

- `HumanFallsImmediatelyJpf:` positions the human ball inside a hole at tick one, so that termination is triggered by a worker through `endGame()`.
- `AllSmallBallsFallTogetherJpf:` positions both small balls inside the holes, so that termination is triggered by the coordinator through `setEndGame()` when `smallBalls.isEmpty()` becomes true.
- `BoundaryCollisionJpf:` places two small balls at distance equal to the sum of their radii, exercising the strict inequality in the collision predicate.
- `ActualCollisionJpf:` places the same two balls overlapping with opposing velocities, so that `resolveCollision` executes its full body and validates the getter fix under concurrent reads and writes from a second worker processing other pairs.

- `HumanAndBotFallSameTickJpf:` places the special balls inside two distinct holes with an odd total ball count, so that `distributeLinearWork` schedules them on different workers, producing concurrent calls to `endGame`.
- `MoreWorkersThanBallsJpf:` configures more workers than balls, leaving one worker idle in `SynchCell.get()` for the entire simulation.

Scenario	**Outcome**	**Time**	**States**	**End**
`PooolJpf` (base)	no errors	13:23	4 456 653	968
`HumanFallsImmediately`	no errors	04:12	1 245 177	968
`AllSmallBallsFallTogether`	no errors	05:50	1 713 266	968
`BoundaryCollision`	no errors	14:39	4 503 405	968
`ActualCollision`	no errors	14:50	4 548 759	968
`HumanAndBotFallSameTick`	no errors	04:22	1 268 577	1 936
`MoreWorkersThanBalls`	no errors	39:23	9 812 120	6 038

Table 3: JPF verification results on `pooolThreadOrientedJpf` with the spatial-grid collision phase. Two workers and two ticks, three workers for `MoreWorkersThanBalls`, with `DeadlockAnalyzer` as listener and `NotDeadlockedProperty` active by default. Times are wall clock.

Every scenario completed without violations, including `MoreWorkersThanBalls`, which had been the only intractable case under the previous collision algorithm. With the earlier flat triangular-matrix resolution that scenario saturated the 2 GB heap of JPF after seven and a half hours and around 94 million new states, and a retry with fewer balls still failed to terminate in practical time. The spatial-grid version completes it in 39 minutes within 440 MB. The gain came entirely from the smaller grid: fewer cells mean fewer shared read-write locations per tick, which is what the verifier's state space counts. Its `end = 6 038`, against 968 elsewhere, reflects the extra termination orderings introduced by the third worker.

One result is worth highlighting. `HumanAndBotFallSameTickJpf` terminated with `end = 1 936`, exactly twice the value observed in all other scenarios. The doubling is direct evidence that JPF explored both orderings of the concurrent `endGame()` calls: human winning the race against bot winning the race, each producing a different `gameResult` string. The result confirms that the `synchronized` declaration on `GameState.endGame` preserves consistency under both orderings, with no intermediate or corrupted state visible.

The suite was run with the `DeadlockAnalyzer` listener. This is sound because a run of the base configuration under `DeadlockAnalyzer` and one under `PreciseRaceDetector` produced the same state count, end count, and maximum depth: the two listeners do not alter the explored state space. Deadlock detection is performed by the default `NotDeadlockedProperty` regardless of the listener, which only enriches the diagnostic output in case of violation. The race fix described above had already been certified under

`PreciseRaceDetector`, and `resolveCollision`, the only place a race was ever found, is unchanged by the grid rewrite. Re-running the suite under the race detector would therefore add cost without coverage.

4.5 Task-oriented Variant and Modelling Limits of JPF

An analogous variant `pooolTaskOrientedJpf` was built to apply the same methodology to the executor-based architecture. A similar shutdown defect was expected, since `Executors.newFixedThreadPool` creates non-daemon threads that the original coordinator never shuts down. JPF refused to execute the variant. At the first `exec.invokeAll(work)` it raised `NoSuchMethodError` on `java.lang.invoke.JPFFieldVarHandle.setRelease` and terminated after six exploration steps. The reason is internal to JPF. The verifier runs on a Java 11 host JVM, but it interprets the bytecode of the program inside its own internal JVM, frozen at version 8 (2014). That internal JVM does not model `VarHandle`, a primitive introduced in Java 9 (2017), and therefore cannot handle the modern implementation of `FutureTask` and `AbstractExecutorService`.

The defect was therefore confirmed by semantic reasoning. A single `exec.shutdown()` call at the end of the coordinator's `run()` was added to the production `pooolTaskOriented`.